

Examen réparti 1

Master 1 SAR

UE 4I401

L. Arantes, P. Cadinot, S. Dubois, J. Lejeune, J. Sopena, P. Sens

Novembre 2018

2 heures – **Tout document papier autorisé**

Tout appareil électronique interdit

Le barème est donné à titre indicatif

Exercice 1 – Questions de cours et callout (5 points)

Question 1 1 point

Pourquoi doit-on interdire la redéfinition du comportement du signal SIGKIL ?

Solution:

Toujours avoir un moyen pour le système de tuer un processus.

Question 2 1 point

Quelle est la différence entre un signal masqué et un signal ignoré ?

Solution:

Ignoré = le signal n'est pas envoyé dans le kill. Masqué = le traitement du signal est retardé jusqu'au demasquage.

Question 3 1½ points

On considère l'Unix du TD avec la table globale `v.ve_callout` de type `struct *callo` qui gère l'ensemble des callout.

```
struct callo
{
    int      c_time;           /* incremental time */
    caddr_t  c_arg;           /* argument to routine */
    int      (*c_func)();     /* routine */
};
```

Initialement la table ne contient aucune fonction. Donnez l'état de la table suite aux 4 appels :

```
timeout(func1, 0,10);
timeout(func2, 0,5);
timeout(func3, 0,20);
timeout(func4, 0, 10);
```

Solution:

func2	0	5
func1	0	5
func4	0	0
func3	0	10
0	-	-

Question 4 1½ points

Donnez l'état de la table 6 ticks après l'appel aux fonctions timeout ?

Solution:

func1	0	4
func4	0	0
func3	0	10
0	-	-

Exercice 2 – Synchronisation (4,5 points)

On souhaite proposer un nouveau mécanisme de synchronisation dans le noyau. La fonction interne au noyau `wait_N(char *ad)` permet à l'appelant d'attendre que N processus aient appelé `wait_N` avec l'adresse `ad`. Dès que les N processus ont fait l'appel, ils sont débloqués. On suppose que N est une constante.

Question 1 2½ points

Donnez le code de la primitive `wait_N(char *ad)` sans ajouter de champ supplémentaire dans la structure `proc`. Pour simplifier on suppose que les `wait_N` sont appelés sur des adresses différentes des autres `sleep`. Les processus s'endorment à la priorité `PWAITN`.

Solution:

```
wait_N(char *ad) {
    register struct proc *p;
    int n = 0;

    for (p = &proc[0]; p < &proc[NPROC]; p++)
        if (rp->p_flag & SSLEEP && p->p_wchan == ad)
            n++;
    if (n == N-1)
        wakeup(ad);
    else
        sleep(ad, PWAITN);

    return;
}
```

Question 2 2 points

On souhaite modifier la fonction `wait_N` pour que la fonction ne se bloque pas plus de t ticks. Modifiez la fonction en conséquence, vous pourrez utiliser les fonctions `timeout` et `untimeout` vues en TD :

```
id = timeout (void (*fun) ( ), void *arg, int tim);
untimeout(int id);
```

La fonction *timeout* renvoie une valeur entière (*id*), qui identifiera l'appel du timeout en question. La fonction *untimeout* enlève de la table de *callout* l'entrée correspondante au *timeout* dont l'identifiant est *id*.

Solution:

```
wait_N(int t, char *ad) {
    register struct proc *p;
    int n = 0, id;

    for (p = &proc[0]; p < &proc [NPROC]; p++)
        if (rp->p_flag&SSLEEP && p->p_wchan == ad)
            n++;
    if (n == 1)
        id = timeout(wakeup, ad, t); // Le premier lance le timeout
    if (n==N-1) {
        untimeout(id); // le dernier annule le timeout
        wakeup(ad);
    }
    else
        sleep(ad, PWAITN);

    return;
}
```

Exercice 3 – Sleep/wakeup (10,5 points)

Nous souhaitons fournir des variantes de sleep (*sleep_session_soft* et *sleep_session_hard*) et wakeup (*wakeup_session*) permettant d'endormir et réveiller un groupe de processus.

Plus précisément, nos variantes de sleep endormiront tous les processus (au moment de l'appel) en état SRUN de la même session que le processus appelant (c'est à dire partageant le même *p_ttyd*) avec la priorité PSESSION<PZERO. La première variante *sleep_session_soft* n'affectera pas les processus de la même session que l'appelant déjà endormis (c'est à dire qu'ils seront réveillés par le wakeup sur leur adresse de sommeil actuelle et non pas par le *wakeup_session*).

La seconde variante *sleep_session_hard* affectera les processus de la même session que l'appelant déjà endormis (c'est à dire qu'ils seront réveillés par le *wakeup_session* et non pas par le wakeup sur leur adresse de sommeil actuelle).

Dans les deux cas, *wakeup_session(n)* réveillera l'ensemble des processus de la session *n* endormis par le(s) appel(s) antérieur(s) à *sleep_session_soft* et *sleep_session_hard* et renverra le nombre de processus réveillés par la fonction. Les prototypes des fonctions sont les suivants :

```
void sleep_session_soft();
void sleep_session_hard();
int wakeup_session(short* ttyd);
```

Question 1 1 point

Quelles sont les implications du choix de la priorité PSESSION ?

Solution:

- priorité au réveil du processus égale à PSESSION
- processus non swappable pendant son sommeil
- processus non interruptible par un signal pendant son sommeil

Barème : 33% par item + 1% bonus pour ceux qui trouvent les 3

Question 2 1½ points

En utilisant la fonction interne `sleep_session_soft`, donnez le code d'un **appel système** `sys_stop_session` permettant d'endormir l'ensemble des processus de la session de l'appelant. Cet appel système renverra systématiquement la valeur 1 après le réveil du processus appelant.

Solution:

```
sys_stop_session() {
    sleep_session_soft();
    u.u_ar0[R0]=1;
}
```

Barème :

- 45% récupération du `ttyd` du processus courant
- 10% appel `sleep_session`
- 45% transmission de la valeur de retour

Question 3 1½ points

En utilisant la fonction interne `wakeup_session`, donnez le code d'un appel système `sys_continue_session` permettant de réveiller tous les processus de la session dont le numéro est donné en paramètre. Cet appel système renverra le nombre de processus réveillés si celui-ci est non nul, la valeur d'erreur `ECONT` sinon.

Solution:

```
sys_continue_session() {
    ttyd=u.u_arg[0];
    nb=wakeup_session(ttyd);

    if(nb>0) {
        u.u_ar0[R0]=nb;
    } else {
        u.u_error=ECONT;
    }
}
```

Barème :

- 45% gestion du paramètre
- 10% appel `wakeup_session`
- 45% gestion de la valeur de retour et du code erreur

Question 4 1½ points

Pour implémenter les fonctions internes `sleep_session_soft`, `sleep_session_hard` et `wake_session`, nous décidons d'ajouter un champ `sleep_session` (1 si le processus est en sommeil de session, 0 sinon) dans le descripteur de tâche. Ce champ sera-t-il placé dans la struct `proc` ou dans la zone `U` ? Pourquoi ? Pourquoi n'est-il pas possible d'utiliser le champ `p_wchan` ?

Solution:

- dans la struct `proc`
- il s'agit d'une information devant être accédée par un autre processus (celui qui va exécuter le `wakeup_session`), il faut donc la placer dans la partie résidente du descripteur de tâche.
- Le champ `p_wchan` est utilisé pour stocker une adresse de sommeil, il pourrait y avoir un conflit entre une adresse et un numéro de session et il serait impossible de distinguer un processus endormis sur `sleep` d'un processus endormis sur `sleep_session_*` dans le `wakeup_session`.

Barème : 33% par item + 1% bonus pour ceux qui trouvent les 3

Question 5 2 points

Donnez le code de la fonction interne wakeup_session(int ttyd).

Solution:

```
int wakeup_session(short * ttyd){
    int n = 0;
    for(int i=0; i<NPROC; i++){
        if(proc[i]->p_ttyd==ttyd&&proc[i]->p_sleep_session){
            setrun(proc[i]);
            n++;
        }
    }
    return n;
}
```

Barème :

- 30% parcours de la table proc
- 30% test
- 30% appel à setrun
- 10% gestion du "n"

Question 6 2 points

Donnez le code de la fonction interne sleep_session_soft().

Solution:

```
void sleep_session_soft(){
    spl(CLINHB);
    for(int i=0; i<NPROC; i++){
        if(proc[i]->p_ttyd==u.u_procp->p_ttyd&&p->p_wchan==0){
            proc[i]->p_sleep_session=1;
            proc[i]->p_stat=SSLEEP;
            proc[i]->p_pri=PSESSION;
        }
    }
    spl(NORMAL);
    swtch();
    return;
}
```

Barème :

- 20% masquage IT autour de la boucle
- 10% parcours de la table proc
- 30% test
- 20% modification struct proc
- 20% swtch correctement placé

Question 7 1 point

Modifiez votre réponse à la question précédente pour obtenir le code de la fonction interne sleep_session_hard().

Solution:

```
void sleep_session_hard(){
    spl(CLINHB);
```

```

for(int i=0;i<NPROC;i++){
    if(proc[i]->p_ttyd==u.u_procp->p_ttyd){
        proc[i]->p_sleep_session=1;
        proc[i]->p_wchan=0;
        proc[i]->p_stat=SSLEEP;
        proc[i]->p_pri=PSESSION;
    }
}
spl(NORMAL);
swtch();
return;
}

```

Barème :

- 50% modification du test
- 50% prise en charge du champ *p_wchan*